

Functions

Amir Jamalian



Gisma
University
of Applied
Sciences





- Functions are groups of related statements that can be called together typically perform a specific task
 - It may or may not take a set of parameters
 - It may or may not return values
- Functions are one way to break an application down into more manageable units
 - That can be worked, tested, and maintained separately
 - Functions can be defined in one place once and called or invoked in multiple locations
 - It makes code more modular and easier to understand
 - A piece of functionality needs to be maintained and tested in only one location
 - As it is only defined once

$$f(x) = x^2$$
$$f(3) = 3^2$$



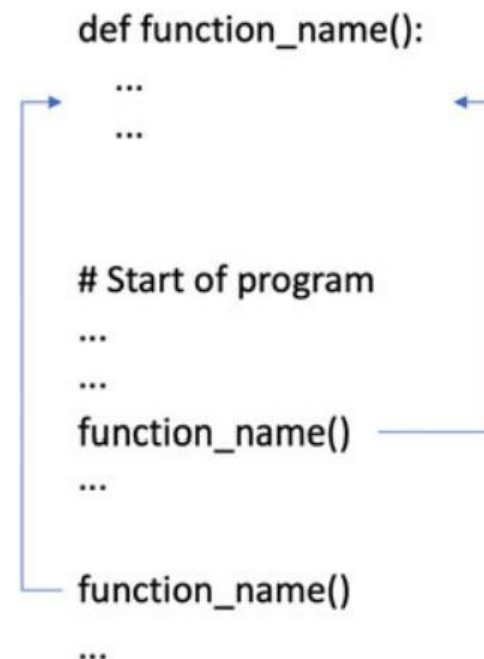
How Functions Work

[Hunt 2019]

3

• When a function is called (or invoked)

- The flow of control a program jumps from where the function was called to the point where the function was defined
 - All the values that were in place when the function was called are stored away
 - On something called the stack
 - If the function defines its own versions of the variables, they do not overwrite each other
- The body of the function is then executed before control returns back to where it was called from





Types of Functions

- Built-in functions

- Are those provided by the language
- We have seen several of these already
 - E.g., `print()` and `abs()`

- User-defined functions

- Are those written by developers
- We have to either import or define them



Importing Functions from Modules

- Most functions that are built into the Python language are stored in a collection of functions called a module
 - Consider a module to be the same as a code library
- An import statement is used to provide access to a module
 - Importing the whole module
 - `import math`
 - `math.exp(2)`
 - Importing only parts of a module
 - `from math import exp`
 - `exp(2)`

$$f(x) = e^x$$
$$f(2) = e^2$$



Defining Functions

- The function header defines the signature of the function
 - I.e., what it is called and the parameters it takes
 - All (named) functions are defined using the keyword `def`
 - The same variable naming conventions also apply to functions
 - The function names should be all lower case with the different elements of the function name separated by an `_`
 - A function can (optionally) have a list of parameters, which allow data to be passed into the function
 - A colon (`:`) is used to mark the end of the function header and the start of the function body
- The function body defines what the function does
 - An optional documentation string (the docstring) can describe what the function does
 - The function might return value(s) using the `return` statement

```
def function_name(parameter list):  
    """docstring"""  
    statement(s)
```



Example: Factorial Function

[Hunt 2019]

7

- Write a factorial function that takes a positive integer and calculates its factorial

- factorial(5) must return 120
 - Because $1 \times 2 \times 3 \times 4 \times 5 = 120$

```
def factorial(n):  
    """  
    This function takes a value n  
    and returns its factorial n!.  
    """  
    result = 1  
    for i in range(1, n + 1):  
        result *= i  
    return result
```



- The documentation can become more important
 - As functions become more complex and may have multiple parameters
- The docstring allows the function to provide some guidance on what is expected
 - The data should be passed into the parameters
 - What will happen if the data is incorrect
 - What the purpose of the function is
 - The docstring can be read directly from the code and also IDE
 - It is even available to the programmer via a special property of the function called `__doc__`
 - `print(factorial.__doc__)`

```
def function_name(parameter list):  
    """docstring"""  
    statement(s)
```



Function Parameters

- Parameters and arguments are different according to the terminology
 - A parameter is a variable defined as part of the function header and is used to make data available within the function itself
 - An argument is the actual value or data passed into the function when it is called
- `def my_function(a, b, c)`
 - We can have any number of parameters separated by comma

```
def function_name(parameter list):  
    """docstring"""  
    statement(s)
```



Default Parameter Values

- `def my_function(a=1, b=True, c="")`
 - We can provide default values for parameters
- `my_function(20, False)` `# a=20, b=False, c=""`
 - If a value is supplied for the parameter, then it will override the default
- Any number of parameters in a function's parameter list can have a default value
 - However, once one parameter has a default value all remaining parameters to the right of that parameter must also have default values



Named Arguments

- If a function has several parameters, it may become impossible to rely on using the position of a value
 - To ensure it is given to the correct parameter
- `my_function(c="Hello", a=1000, b=True)`
 - We can use named arguments (or keyword arguments) To provide the name of the parameter we want an argument/value to be assigned to
- The Position is no longer relevant
 - However, you cannot place positional arguments after a named argument



Arbitrary Arguments

- Sometimes, we do not know how many arguments will be supplied when a function is called
 - Python allows us to pass an arbitrary number of arguments into a function and then process those arguments inside the function
- A parameter is marked with an asterisk (*)
 - To define a parameter list as being of arbitrary length

```
def greeter(*args):  
    for name in args:  
        print("Welcome", name)
```

```
greeter("John")  
greeter("John", "Alice")
```



Anonymous Functions

- Sometimes, we want to create a function and use it only once
 - Giving it a name for this one time can pollute the namespace of the program
 - There would be a lots of names around
 - Someone might call it when we do not expect them to
- An anonymous function does not have a name and can only be used at the point that it is defined
 - Anonymous functions are defined using the keyword lambda
 - That is why they are also known as lambda functions
 - Anonymous functions can have any number of arguments but only one expression as their body
 - The expression returns a value

Lambda arguments: expression

*Lambda i: i * i*



Recursion

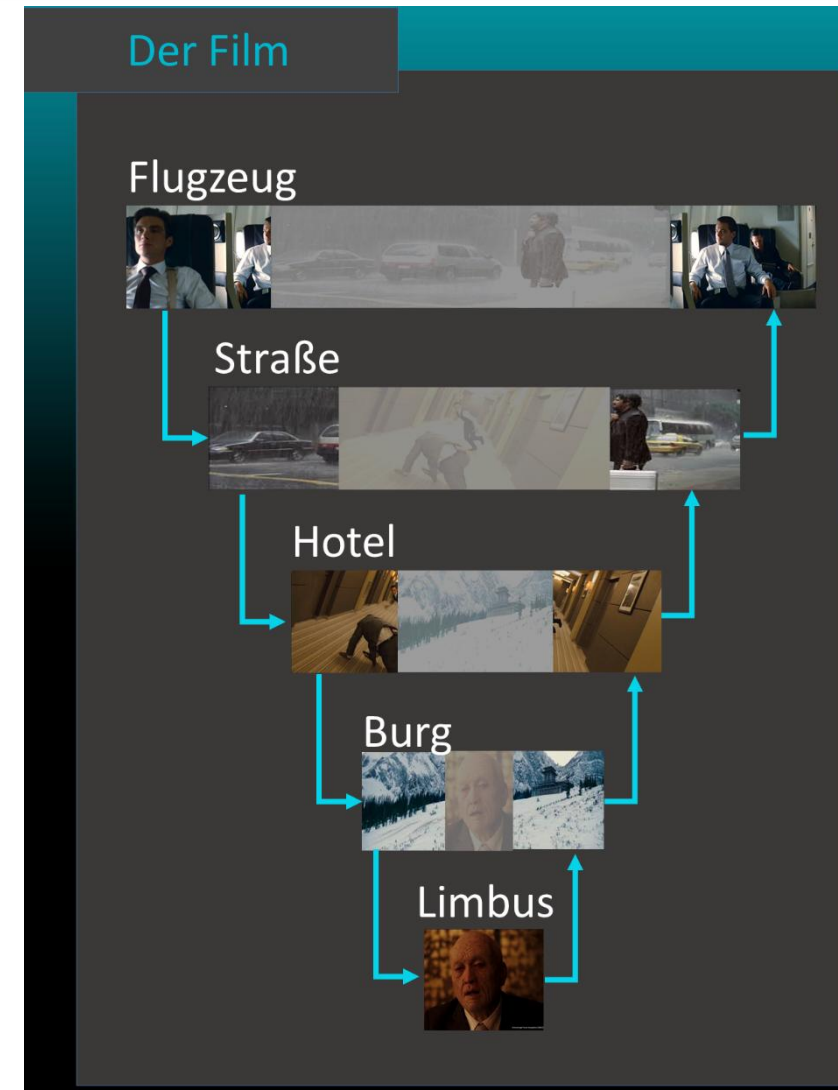
- Recursion is a very powerful way to implement solutions to a certain class of problems
 - Where the overall solution to a problem can be generated by breaking that overall problem down into smaller instances of the same problem
 - The overall result is then generated by combining together the results obtained for the smaller problems
- A recursive function calls itself one or more times in order to solve a particular problem
 - For a recursive function to be useful it must therefore have a termination condition
 - To avoid calling itself to infinity (at least in theory)



Recursion

- The recursive dream function in Inception

```
def dream(location):  
    if location == "Limbo":  
        return  
    return  
    dream(location.next())
```





Example: Recursive Factorial Function

[Hunt 2019]

18

- Write a factorial function that takes a positive integer and calculates its factorial

- factorial(5) must return 120

- Two approaches

- $n! = 1 \times 2 \times \dots \times n$
 - $n! = \begin{cases} 1 & n = 1 \\ n(n-1)! & n > 1 \end{cases}$

```
def factorial(n):  
    result = 1  
    for i in range(1, n + 1):  
        result *= i  
    return result
```

```
def recursive_factorial(n):  
    if n == 1:    # The termination condition  
        return 1    # The base case  
    return n * recursive_factorial(n - 1)    # The recursive call
```



Example: Recursive Factorial Function

[Hunt 2019]

19

- $rf(3)$
- $3 * rf(2)$
- $2 * rf(1)$
- 1

$$\left. \begin{array}{l} rf(3) \\ 3 * rf(2) \\ 2 * rf(1) \\ 1 \end{array} \right\} rf(3) = 3 * 2 * 1 = 6$$

```
def rf(n):  
    if n == 1:  
        return 1  
    return n * rf(n - 1)
```



Scope and Lifetime of Variables

- The scope of a variable is the part of a program where the variable is known
 - Parameters and variables defined inside a function are not visible from outside
 - Hence, they have a local scope
 - The lifetime of a variable is the period throughout which the variable exists in the memory of your Python program
 - The lifetime of variables inside a function is as long as the function executes
 - These local variables are destroyed as soon as the function returns or terminates
- Local variables are scoped only to that function
 - They are not accessible or visible outside of the function
 - It will generate an error
 - If we attempt to access a local variable outside the function

```
def my_function():  
    a_local_variable = 100  
    print(a_local_variable)
```

```
# This line generates an error  
print(a_local_variable)
```



Global and Local Variables

- Global variables are (potentially) accessible anywhere (or globally) in our programs
 - Developers usually try to limit the number of global variables in their programs as global variables can be accessed anywhere and can be modified anywhere
 - This can result in unexpected behaviours

```
x = 100
```

```
def my_function():  
    print(x)
```

```
my_function()
```



The Global Keyword

[Hunt 2019]

23

- We can reference the global variable within a function

- However, we cannot modify the global variable inside the function
 - Python thinks we are creating a local variable
 - To protect us from ourselves, Python is really saying 'Do you really want to modify a global variable here?'

- We need to use the keyword `global` with the name of the variable

- To tell Python that we know what we are doing

```
x = 100
```

```
def my_function():  
    print(x)
```

```
my_function()
```

```
x = 100
```

```
def my_function():  
    global x  
    x = 0  
    print(x)
```

```
my_function()
```



The Nonlocal Keyword

[Hunt 2019]

24

- It is possible to define functions inside other functions

- However, local variables are local to a specific function
 - Even functions defined within another function cannot modify the outer functions local variables
 - As the inner function is a separate function
 - The global keyword is no help here as the outer function's variables are not global
 - They are local to a function

- We can use the nonlocal keyword

- That indicates a variable is not global but is also not local to the current function
- Python should look within the scope in which the function is defined to find a local variable with the same name

```
def outer():  
    title = "original title"  
  
    def inner():  
        nonlocal title  
        title = "another title"  
        print("inner:", title)  
  
    inner()  
    print("outer:", title)  
  
outer()
```